From Fields to Dose: System Architecture

Software, data model, pipeline, operations and release plan for the study described in From Fields to Dose: Study Report.

Author record: https://orcid.org/0009-0005-3459-0859

Repositories: a private working repository and a public release repository (from-fields-to-dose on GitHub) regenerated from it by script (Section 7). Local working copy on the author's machine under a Python virtual environment. Language: Python 3. Secrets: a git-ignored .env file holding provider keys; config/models.yaml holds environment variable names only and is safe to publish.

Writing standard: no em or en dashes; no clock times; hyphens only inside compound terms.

1. Design principles

1. Nothing that affects a result is editable after the lock without a logged deviation: the test set, the scorer, the calculator, the prompts, the output schema and the registered analysis script are hashed together, and the scorer refuses to run if any of them changed.

2. Every number in the paper is produced by one committed script from committed inputs. No hand computation.

3. Every provider is accessed through the same OpenAI-compatible client with a pinned model identifier; the only per-provider differences are endpoint and key name.

4. Runs are resumable and idempotent at the record level: interrupting and restarting never loses or duplicates a scored record.

5. The evaluated models never share a family with the generator and auditor, enforced in code.

6. All artefacts are releasable: synthetic data, prompts, outputs, logs, configuration without secrets.

2. Repository layout

```
f2d/

  config/models.yaml            evaluated models, generator, auditor, budget ceiling, repeat
settings

  sources/                      dosing table, FDA labels, growth references, age bands, source
checks

    dosing_sources.csv          22 verified rows, 21 drugs, label set ids, bases, ranges, maxima

    labels/<drug>/              openFDA label JSON per drug

    wtageinf.csv, wtage.csv     WHO 0 to 24 months and CDC 2 to 20 years LMS tables

    age_bands.json              NICHD bands

    fetch_labels.py, check_sources.py
```

```
  generator/                    record synthesis
    build_corpus.py             scenario and record generation, weight sampling, distractors, dates
    templates.py                note templates by complexity level, frequency vocabulary
    calculator.py               deterministic exposure calculator (locked)
    audit.py                    realism audit with a committed rubric
  harness/                      model execution
    prompts/system.txt, task_B.txt, task_C.txt   committed prompts (locked)
    output_schema.json          committed output object (locked)
    common.py                   client construction, retry policy, JSON extraction, family check
    runner.py                   resumable runner for a split, workflow, run id, optional model and
subset
  scoring/
    score.py                    four-level scoring, abstention scoring, automatic error classes
(locked)
    codebook.md                 manual coding classes and procedure
  analysis/
    analyze.py                  registered primary analysis, clustered bootstrap, hypothesis tests
(locked)
    analyze_malformed_as_failure.py   the registered analysis with the malformed-row correction
(deviation; outside the lock)
    stability.py                run-to-run agreement on the repeat subset
    pairwise.py                 Holm-corrected pairwise model contrasts (added after registration)
  tools/
    lock_testset.py             writes LOCK.json; hash_file and hash_scoring used by the scorer's
check
    self_audit.py, remove_records.py, make_repeat_subset.py
    code_errors.py              blinded manual coding and re-coding
    per_drug.py, admin_amount.py, by_stratum.py   supplementary and exploratory tables (added after
registration)
    export_results.py           export of summary, tables, manifest, lock, deviations, sanitised
config
    ping_models.py, smoke.py, secret_scan.py, show_dosing.py, set_row.py, fetch_query.py
  data/dev/                     development split (172 records)
  data/test/                    locked test split (361 records), LOCK.json, repeat_subset.txt,
audits
```

```
  protocol/                     Protocol v1.0 (docx and PDF), LOCK_hashes.txt, DEVIATIONS.md,
README.md, archive/

  results/                      raw and scored run files, analysis outputs, manual codes, logs/,
export/

  tests/                        unit tests (25)

  SETUP_GUIDE.md                BLOCK 0 to BLOCK 8 operating procedure
```

3. Data model

3.1 Scenario (data/test/scenarios.jsonl). One canonical regimen: scenario id; drug; dosing basis (mg/kg/dose, mg/kg/day, mcg/kg/dose, mcg/kg/day); weight-normalised dose; frequency per day; route; concentration; maxima as max_per_dose_mg or max_per_day_mg; expected per-dose amount (printed, rounded, maximum applied) and expected mg/kg per dose and per day; age band; complexity; required fields; missing fields; conflict fields; challenge tags (for example derive_per_dose, multiple_weights, discontinued_same_drug, lb_documentation, multiple_meds, dose_cap, per_day_basis, mcg_drug, weight_tail); abstention flag.

3.2 Record (data/test/records.jsonl). One note instance of a scenario: record id (pattern test-<drug>-<basis>-<band>-L<level>-v<variant>-r<round>); scenario id; split; complexity; age band; generation method (template or paraphrase); abstention flag; text.

3.3 Raw output (results/raw_test_<workflow>_<model>_<run>.jsonl). First line: a _config object (model id, endpoint, parameters, prompt hashes, split, workflow, run id). Then one line per attempt: record id, scenario id, workflow, run id, raw text, parsed object or null, error or null, retries, latency, timestamp. A record may appear more than once if a run was resumed; the last line is authoritative.

3.4 Scored row (results/scored_test_<workflow>_<model>_<run>.jsonl). Record id, scenario id, workflow, run id, age band, complexity, abstention flag, conflict flag, challenges, generation method, malformed flag, per-field correctness, field accuracy, complete regimen, automatic codes, observed mg/kg per dose and per day, deviations, dosing consistent, dose consequential (10 percent) and sensitivity (20 percent), exceeds labelled maximum (per-day maxima only), primary endpoint.

3.5 Manual codes (results/manual_codes.jsonl, results/manual_codes_recode.jsonl). One line per coded residual error: source file, record id, note text, parsed output, automatic codes, manual code; the re-code file adds the second code. A discarded assisted pass is retained as results/manual_codes_assisted_pass_discarded.jsonl.

3.6 Supplementary tables (added after registration). results/per_drug_run1.csv; results/admin_amount_run1.csv (one row per record: reference printed amount, written amount, deviation, status); results/by_stratum_run1.csv (long format: stratum kind, stratum, model, workflow, n, primary endpoint rate, composite failure rate).

4. Pipeline

4.1 Sources. fetch_labels.py pulls one label per drug from openFDA (query overrides for salts such as amoxicillin and clavulanate potassium). The author verified each row of dosing_sources.csv against the label text using show_dosing.py and set_row.py; the notes column records which maxima are stated by the label and which are the author's derivations. check_sources.py enforces completeness and flags a drug carrying both per-dose and per-day rows (gentamicin; accepted and logged).

4.2 Generation. build_corpus.py samples age within band, sex, and weight from the LMS tables (z in minus 2 to plus 2; 10 percent tails at z 2.05 to 2.5; cap 2.2 times the median), chooses a drug permitted for the age, draws a dose within the label range, and renders a note from templates.py at the requested complexity. Distractors: a second medication (cholecalciferol under 6 months, cetirizine otherwise); a prior-visit weight dated relative to age so it is never before birth; a discontinued order for the same drug; pounds in some level-3 notes; computed per-dose amounts and volumes in level-2 and level-3 notes, withheld in 36 level-3 scenarios. Abstention cases withhold the interval or the current weight. A fraction of records is paraphrased by qwen2.5:14b and audited by the same model with a committed rubric (audit.py); flagged records are reviewed by the author.

4.3 Lock. lock_testset.py writes data/test/LOCK.json with the record count, SHA-256 of records.jsonl, scenarios.jsonl and repeat_subset.txt as raw bytes, and scoring_sha256 computed by hash_scoring() as one SHA-256 over the concatenated bytes of seven files: scoring/score.py, generator/calculator.py, harness/prompts/system.txt, harness/prompts/task_B.txt, harness/prompts/task_C.txt, harness/output_schema.json and analysis/analyze.py. The runner refuses to start a test run without the lock file; scoring/score.py check_lock() recomputes the records hash and hash_scoring() on every scoring run and exits if either differs. The hashes are in protocol/LOCK_hashes.txt and in the OSF registration. Verification commands: shasum -a 256 on the three data files; python3 -c "from tools.lock_testset import hash_scoring; print(hash_scoring())"; git log 528c2b5..HEAD on the seven files, which should list no commits.

4.4 Execution. Workflows are named B (end-to-end) and C (hybrid) as registered; the registered rules-only Workflow A was not run (deviations log) and the surviving letters are kept because they are fixed in the registration, in the hashed prompt file names task_B.txt and task_C.txt, and in every raw and scored file name and workflow field. runner.py takes split, workflow, run id, optional model and subset. For each model it reads existing output, marks records with a parsed object as done, and queries the rest. Each call uses the committed system prompt, the workflow task prompt and the record text; the response is parsed with extract_json; a non-JSON response triggers one repair message; a still-unparseable response is stored with parsed null. Transient errors are retried; billing, credit, quota and rate-limit errors stop that model's loop with a message to rerun later. Workflow B asks the model to compute dose_per_admin_mg and to report mg_per_kg_per_dose and mg_per_kg_per_day; workflow C asks for the fields only.

4.5 Scoring. score.py reads each raw file, keeps the last attempt per record, joins the scenario, normalises units, compares fields at 1 percent tolerance (dose_per_admin_mg not required when the note withholds it), scores abstention cases, and assigns automatic error classes by rules. Levels 3 and 4: in workflow B the observed mg/kg per dose and per day are the model's reported fields; in workflow C they are computed by generator/calculator.py from current weight, dose basis, weight-normalised dose, dose unit, frequency and maxima. The composite dose_consequential is True when the worse of the two relative deviations exceeds 10 percent or when either observed value is missing; the 20 percent sensitivity and the per-day labelled-maximum flag are computed alongside. The written per-dose amount does not enter Level 4 in either workflow. A parsed null is a malformed output and scores incorrect on all fields.

4.6 Analysis. analyze.py (locked) takes the run1 scored files and reports, per model and workflow: primary endpoint, field accuracy, complete regimen, composite failure rate with clustered bootstrap CIs (2,000 resamples over scenarios); abstention metrics; breakdowns by complexity, band and generation method; malformed counts; top error codes; and the H3 paired comparison on the composite rate with its CI and verdict. analyze_malformed_as_failure.py is the same script with malformed non-abstention rows counted as failures

(logged deviation); it is used for any file containing malformed rows and gives identical output on run1. stability.py takes three runs of one model and workflow and reports agreement on the repeat subset. pairwise.py computes Holm-corrected paired contrasts between models on the primary endpoint and, with its load and paired_boot functions, the H3 difference on the primary endpoint (Protocol Section 16 wording). per_drug.py, admin_amount.py and by_stratum.py write the supplementary tables in 3.6.

4.7 Manual coding. code_errors.py collects residual extraction errors from the scored files, shuffles them with a fixed seed, hides the model name, shows the note, the output and the automatic codes, and appends each typed code to results/manual_codes.jsonl immediately. Quitting and rerunning resumes. The --recode option draws a 10 percent sample with a second seed, presents it again, and writes agreement. The author codes alone; no assistant suggests a code for any item.

4.8 Export. export_results.py writes results/export/ with summary.json (every quoted number), tables as CSV, the run manifest (model ids, dates, settings, prompt hashes, cost), LOCK.json, DEVIATIONS.md and a sanitised configuration.

5. Integrity controls

Control	Mechanism
Test set immutability	SHA-256 in LOCK.json, published on OSF; runner and scorer refuse on mismatch
Scorer, calculator, prompt, schema and registered-analysis immutability	One combined hash at lock; the scorer recomputes it on every run; any change requires a logged deviation; a working-copy edit to the analysis script on 5 October was caught by this check, reverted, and moved outside the hashed set
Model pinning	Exact identifiers in config/models.yaml and in each raw file's _config line
Generator and evaluated model separation	Family check in harness/common.py refuses overlap
Resumability	Done set keyed on parsed output; last attempt wins at scoring; identical for all models
Blinding of manual coding	Model name hidden; randomised order; fixed seed; append-only file; unassisted
Secrets	.env git-ignored; secret_scan.py before any push, including results/logs; keys rotated after the study
Budget	200 USD ceiling in config; spend recorded per provider in the manifest
Deviations	protocol/DEVIATIONS.md, one dated row per change with rationale

6. Operations

6.1 Standard sequence (SETUP_GUIDE BLOCKS 0 to 8). 0 environment and keys; 1 tests and smoke; 2 sources; 3 development corpus and runs; 4 test corpus, audits, self-audit, lock; 5 protocol finalisation and OSF registration; 6 test runs (run1 both workflows, repeats run2 and run3 on the subset); 7 scoring, analysis, stability, manual coding; 8 export and hand-off.

6.2 Running and resuming. One background job per chain (caffeinate plus nohup); progress read from the output files by counting records with parsed output; any interrupted command is rerun unchanged to resume.

6.3 Provider quotas observed. Gemini 3.1 Pro preview: 250 requests per model per Google Cloud project per day on the paid tier; worked around with a second project under the same billing account and a key swap in .env

(both keys retained, one active; logged as a deviation). OpenAI and Anthropic: prepaid credit exhaustion stops the runner with a 429 or 400; top up and rerun. DeepSeek: 402 on insufficient balance; two repeat-run records returned empty responses on every attempt and are scored malformed.

6.4 Observed per-call cost. GPT-5.5 about 3.1 cents; Gemini about 2.2 cents; Claude about 1.0 cent; DeepSeek about 0.9 cents; Llama local. Full study under 100 USD.

6.5 Verification commands used for the report. Lock: 4.3 above. Row-level traces: a short script that joins a record's note text, scenario fields, the model's last parsed output and its scored row (used for the four maximum-dose notes and the neonatal rounding record). Reconciliation: a cross-tab of admin_amount_run1.csv status against the scored composite and its component, with the automatic codes among the disagreeing rows.

7. Public release

7.1 What is public. tools/make_public_release.py copies an allow-list from the working repository into a fresh directory: README, licences and CITATION.cff; config/models.yaml (environment-variable names only); sources/ (dosing table, label JSON, growth references, age bands, source scripts); generator/, harness/ (prompts, schema, runner, client), scoring/, analysis/, tools/, tests/; data/test/ (locked split, LOCK.json, repeat subset, audits) and data/dev/; protocol/ (Protocol v1.0 docx and PDF, LOCK_hashes.txt, DEVIATIONS.md, README) without the archive; results/ raw and scored run files, analysis outputs, supplementary CSVs, manual coding files and the evidence packet, without logs and backups; docs/ with the official documents and figures; SETUP_GUIDE.md.

7.2 What is not public. .env and any environment file, the virtual environment, caches, .DS_Store, results/logs, results/backup, protocol/archive, and any file not on the allow-list. The release directory has its own git history with no connection to the working repository's history, so nothing excluded can be recovered from it.

7.3 Procedure. Run the release script; run tools/secret_scan.py and a pattern scan over the release directory; initialise a fresh repository, commit, push to the public remote; tag the commit; record the tag in the paper. Repeat at each milestone (fifth model complete; coding complete; acceptance). At acceptance: update the OSF project with the paper DOI and the repository URL; rotate all provider keys.

8. Known limitations of the implementation

Single author operates every stage; the realism auditor and paraphrase generator are the same model family; the prompt omits a staleness rule for documented weights (now a reported finding); workflow B's Level 4 trusts the model's reported mg/kg values, so a model that copies the printed amount correctly can still fail on its own inconsistent mg/kg fields, which is why the written amount is reported separately; the per-day maximum flag does not test per-dose maxima; one test-set record has a printed amount more than 10 percent from the exact product; preview model identifiers may be withdrawn by vendors; local inference for Llama is deterministic at the chosen settings, so its stability figure is not comparable to API models; resumed runs give unparsed outputs an extra attempt, applied uniformly and reported.

Repository

Code, synthetic data, locked test split, protocol, raw and scored outputs, analysis outputs, evidence packet, figures and documents: https://github.com/VinodRufus/from-fields-to-dose